Deep Learning-Based Word Translation Pipeline for Videos on Kathakali Dance

Kathakali is an Indian Classical dance form from the Southern Indian state of Kerala, where words used in daily life can be represented using various combinations of arm postures, body angles and finger positions. The video clips in Kathakali consist of keyframes representing still postures, which can be used to describe the full content of a video. In this work, we propose a pipeline to translate Kathakali video clips into the words they depict, which can help in the preservation of the intangible heritage of India by digitally preserving ancient art forms. We recorded dance movements (representing the words) of a single Kathakali artist, with ten videos for each word, and manually identified the keyframes belonging to each word and separated them into separate classes. We extract the keyframes from these video clips, perform pose estimation using DWPose, normalise these pose estimated skeletal images, and extract 66 pose features from these images. We then use a novel deep learning model consisting of a CNN module as well as a Pose module to classify these skeletal images into separate classes. Our fusion model gives an image classification accuracy of 82.77% on 123 classes. We then perform video classification by applying majority voting to the classified keyframes, using a normalised probability calculation as a tie-breaker, and achieve a final video classification accuracy of 97.56%. To test the generalizability of our pipeline, we applied it to a few videos of different people we collected. Finally, we propose our full pipeline as a video-to-word prediction algorithm.

Additional Key Words and Phrases: Kathakali, Keyframes, DWPose, Classification, Pipeline, Algorithm

ACM Reference Format:

. 2026. Deep Learning-Based Word Translation Pipeline for Videos on Kathakali Dance. *ACM J. Comput. Cult. Herit.* 1, 1 (September 2026), 24 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

Kathakali is an Indian Classical Dance (ICD) originating from the state of Kerala, in Southern India. Like every ICD, a Kathakali performance usually has a storyline taken from epics and folklores from Indian subcontinent. This storyline known as *Attakatha* is represented in form of verses. In a Kathakali performance, the verses is sung by the singers, which is enacted by the performers along with accompanying percussion instruments. The *Attakatha* is written in *Manipravalam*, which is a medieval literary style and hybrid language. It has blends of Sanskrit, Malayalam and Tamil, which are languages from India. The building blocks of a Kathakali performance include the *Attakatha* (storyline), and the artists on the stage which includes the actors, the singers, and the percussion instrument players. Figure 1 shows a Kathakali performance with an actor wearing makeup and costume on the stage. The singer is shown in the background, and the musical instrument players are standing on the sides.

Every word in the verses sung by the singers is enacted by the performers (dancers). An actor enacts the scene using their facial expressions (*Navarasams*), hand gestures (*Mudras*), arm movements, leg movements and upper body movements. Kathakali can be considered a complete sign language, as it has its own grammar and syntax which is comparable with that of natural languages. As per Hasta Lakshana Deepika [4], there are twenty-four *Mudras* in Kathakali, which are listed in Table 5 in Appendix A. Figure 9 in Appendix A shows these twenty-four *Mudrasz*. *Mudras* can be considered as the alphabets in this Kathakali sign language since every word is represented using at least one *Mudra*.

Author's Contact Information:

Fig. 1. A Kathakali performance on stage with an actor in front with costume. The singer is shown in the background, and the musical instrument players are standing on the sides. Photograph by Shagil Kannur (2024), licensed under CC BY-SA 3.0 https://creativecommons.org/licenses/by-sa/3.0/ [29]

There are multiple aspects of a Kathakali performance, such as *Attakatha*, music, rhythm and the interpretation of the Kathakali sign language. Upon discussion with Kathakali exponents, we identified that the *Mudras* of the hand, along with the arm movement and position, determine the specific word that is being enacted. In this work, we attempt to transliterate a word being enacted using only the *Mudras* and upper body movements. For example, consider Figure 2, which shows the word *Acchan* (which means Father) being represented using different *Mudras* in different hands. In the initial posture (corresponding to the first image), the left hand *Mudra* is *Mushti* and the right hand *Mudra* is *Hamsapaksham*. In the final posture (corresponding to the second image), the left hand *Mudra* remains *Mushti* while the right hand *Mudra* has changed to *Mudraakhyam*. We try to achieve this using dance posture classification.

Indian Dance posture classification is not an area that has been explored extensively, and only a few works exist in this area [46]. Digitally archiving as well as classifying traditional and cultural dance forms, which consist of facial expressions and body movements, helps to conserve the intangible cultural heritage of a country. This is particularly true in India, a country with a diverse background, featuring varying cultures, languages, and art forms across different regions. This can also help support dance video generation and dance video interpretation. Furthermore, the framework can be applied to video summarisation, efficient indexing and retrieval of dance sequences, and the development of intelligent tools for learning and analysis of classical dance.

Despite the above potential research areas, ICD classification presents various challenges. Few dance video datasets exist, with the same word being represented by more than one posture. Differences in body shape as well as colour can strongly affect the model, leading to bias in training the model. Differences may also exist in body proportions, making limb lengths vary across people. These challenges establish the need for an efficient method for performing dance translation.

In this work, we present a pipeline to translate small video clips of Kathakali performance to natural language using image classification methods by means of deep learning models. We first collect ten videos each of forty-one words

Fig. 2. Word *Acchan* being represented using Mudras *Mushti*, *Hamsapaksham*, and *Mudrakhyam*. In the initial posture corresponding to the first image, the left hand *Mudra* is *Mushti* and the right hand *Mudra* is *Hamsapaksham*. In the final posture corresponding to the second image, the left hand *Mudra* remains *Mushti* while the right hand *Mudra* has changed to *Mudraakhyam*.

used in Kathakali and extract keyframes from them. Each sets of keyframes for a particular word, from all the videos, are grouped and separated into different classes. We then apply DWPose, a pose estimation method, to get skeletal frames from our keyframes, and normalise them using translation and scaling transforms. We then extract three types of pose features from these images, perform correlation analysis on them, and remove the highly correlated features. We further classify the skeletal frames into separate classes using a novel deep learning model consisting of CNN layers as well as pose features. We then use majority voting from the classified keyframes in order to predict the words the videos belong to and, as a tie-breaker, use a normalised probability scoring method for word prediction. To test the generalizability of our pipeline, we test it on a few videos of different people. Finally, we propose our full pipeline as a video-to-word prediction algorithm.

The novelty in our work is fourfold:

(1) Creation of a dataset on Kathakali dance postures corresponding to forty-one words, each consisting of ten videos
(2) Creation of a novel deep learning model that consists of a CNN module as well as a Pose module to perform image classification on the skeletal keyframes
(3) A video classification approach on the classified skeletal keyframes using majority voting and normalised probability scores
(4) Proposal of a novel algorithm based on our pipeline for video-to-word prediction

The rest of the work is divided as follows. Section II consists of the related works in image and video classification, sign language translation, and posture recognition, as well as their limitations. Section III presents the detailed pipeline, including the video collection, keyframe extraction, pose estimation and normalisation, feature extraction and correlation analysis, image classification, and video classification. Section IV shows the results of our work, which include image classification, ablation studies, video classification, as well as the proposed algorithm. Finally, we conclude with Section V.

2 Related Works

Most of the previous works on sign language translation or video classification are performed using AI-based models, with most of them using pre-trained models or creating their own CNN models. The addition of features from images as extra layers of input is present in a few classification approaches [27] [2], though it is not extensively found in sign language translation. Works involving classification of dance postures from across the world are few, and works in the ICD domain are even fewer.

A few works have performed surveys on image and video classification approaches [50] [10], whereas others have focused on classification after examining pooling architectures [61]. Some works have created their own CNN models for various use cases, such as for online shopping [32], breast cancer detection [14], and land cover hyper-spectral images [58]. However, existing CNN models have also been used in classification, usually in combination with each other [13] or using ensemble techniques [39]. Image classification can also be performed without CNN models, such as using colour histograms [12], or LLMs [53].

Works on the classification of Italian monuments [2], disaster-affected heritage sites [31] or European Renaissance paintings [5] fall under the classification of tangible cultural heritage, while efforts have been made to classify and preserve other types of intangible cultural heritage, such as paper cutting and shadow puppetry from China [33]. Previous works on classification of dance postures from around the world include classification of Russian ballet [45] or Greek dances using Machine learning and Deep learning approaches [6], as well as Chinese dances [30] using high-and-low-resolution features. Surveys on classification approaches have been done and tested on eight ICDs [19]. Out of the ICDs, classification of Bharatanatyam dance is the most common [7] [57], though it can be done on Indian folk dances as well [55]. Though Kathakali dance posture classification is not an area that has been explored extensively, works on the classification of *Navarasams* exist, using CNN [48], as well as using a combination of CNNs with algorithmic approaches [26].

Most of the previous works on sign language translation have employed usage of deep learning models, including CNNs. Previous works have included classification on the WLASL dataset using CNNs [1] as well as Spiking Neural Networks [11]. CNNs can be used for ASL (American Sign Language) translation [42], as well as Arabic Sign Language translation [8] [17]. It is also used for CSL (Chinese Sign Language) translation [44], though when it comes to works involving multiple sign languages, LSTMs are more commonly used [15] [3]. Works on translation of ISL (Indian Sign Language) also exist, using CNNs [25] as well as LSTMs [51]

Previous works on posture recognition have measured the angles of joints, developed algorithms, or used deep learning models. Body posture classification can be done by directly detecting the angles [28], or by using a combination of angles and pressure points [41]. Algorithms can also be developed for posture recognition [23] [34]; however, most of the work has concentrated on the development of CNN models for body posture detection. CNN models developed for posture detection include transformer models [35], Pyramid Graph CNNs [62], attention mechanisms [49], as well as advanced object detection models like YOLOv11 [63]. Works also involve the use of pose estimation methods like OpenPose [21] or MediaPipe [38].

Even though all the previous works have gone deeply into different video classification methods, sign language translation and pose estimation, a few limitations exist across these works. The number of works on pose detection in dance videos is rare, and in the ICD domain even rarer. Many existing works do not include keypoint normalisation and can be used only with specific datasets. Pose estimation methods, such as OpenPose or MediaPipe, though good

for full-body postures, cannot identify the keypoints across fingers properly. This is a major limitation in areas such as Kathakali, where wrong identification of finger keypoints can lead to incorrect word prediction.

Considering these issues in mind, we propose a novel word translation pipeline that automatically detects the keyframes, creates normalised skeletal keyframes and extracts pose features, performs image classification, and accomplishes video classification by classifying the separate images in a video. The addition of extra pose features from the images only enhances the accuracy of the created model, and the usage of better pose estimation models further enhances the visualisation of the finger keypoints as well as improves the generalizability of our model. Finally, our pipeline gives better accuracy than the existing video classification methods, further proving its effectiveness. We also propose an algorithm for video-to-word translation, based on the developed pipeline.

3 Methodology

The detailed Word Translation pipeline is shown in Figure 3 and is split into the six steps given below. These six steps are detailed in the following subsections.

- Video dataset collection
- Keyframe extraction
- Pose estimation and Normalization
- Feature extraction and Correlation analysis
- Image classification
- Video classification

3.1 Video Dataset collection

Our first task was to collect Kathakali dance videos. We used forty Malayalam words commonly used across various plays in Kathakali for our classification. These Malayalam words in their English transliterations and English translations, are shown in Table 1, sorted alphabetically according to their English transliterations. We observed that ambiguities present in languages can also arise in Kathakali Sign language. One such example is the word *Nee* (English meaning "You"), which has two different types of representations, using two different Navarasams (facial expressions), *Shringaaram* and *Roudram*. We decided to use both representations and, to avoid confusion, viewed them as separate words, bringing the total to forty-one words.

We recorded the dance movements (representing the words) of a single Kathakali artist in two different backgrounds. Ten videos for each of the forty words were recorded with small changes in the artist's position. The data were collected in 1080p as well as 4K resolutions, with some videos being recorded at 30 fps and some at 60 fps. All the videos were recorded on an iPhone 17. These settings helped us ensure that the videos emulated real-world data with different resolutions. Each video was around 5-10 seconds long and was named in the format <Word>_<number>. For example, if the fifth iteration of the word *Aakasham* (English meaning Sky) was being recorded, it was named as *Aakasham_5*. The videos using the word *Nee* and the *Navarasam Shringaaram* were named as *Nee_Shringaaram_1*, *Nee_Shringaaram_2*, etc. and a similar approach was followed for *Nee* with *Roudram*.

3.2 Automatic Keyframe Extraction

Our next task was to identify and name the keyframes from the videos recorded. The dance postures in Kathakali can be classified into key postures and transition postures. Key postures are postures where human body movement is

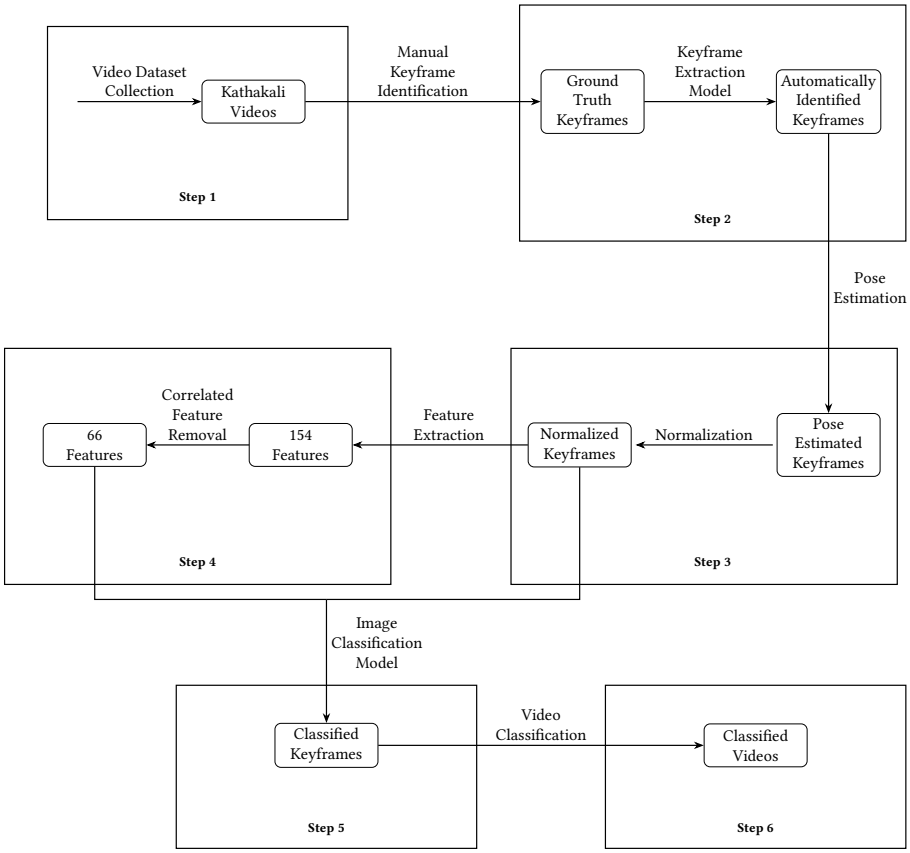

Fig. 3. Proposed Video Classification Pipeline

minimal, whereas some human body movement happens in Transition Postures [40]. Corresponding to these postures, the frames in a Kathakali dance video can also be divided into keyframes and transition frames. Since transition frames consist of movements, most of these frames are blurred and therefore do not contribute much to the classification. Therefore, in our approach, we ignore the transition frames and concentrate only on the keyframes.

To extract the keyframes, we first extracted all the frames from the videos and named them. We followed the naming convention <word>_<iteration>_<frame_number> to name the frames. For instance, if the video currently being considered was *Aana_1* (where *Aana* is the word and 1 is the iteration), then the frames were named as *Aana_1_001*, *Aana_1_002*, etc.

3.2.1 Manual Keyframe (Ground Truth) Extraction. Our next task was to manually identify the keyframes from all the frames in all the videos. Since the transition frames consist of blurry data, we disregard them and only use the keyframes. We divided the video into three parts: the starting pose, the middle pose, and the end pose. Corresponding to each part, a set of keyframes was extracted. For instance, if the three key positions from the video *Aakasham_1* correspond to the frame numbers 1-24, 81-112, and 173-199, then the frames *Aakasham_1_001* to *Aakasham_1_024*,

Table 1. Words used

Malayalam words	English meanings	Malayalam words	English meanings
Aakasham	Sky	*Manassu*	Mind
Aana	Elephant	*Nadakkuka*	To walk
Aayudham	Weapon	*Nee*	You
Acchan	Father	*Paadam*	Feet
Atbhutham	Wonder	*Pazham*	Banana
Amma	Mother	*Pushpam*	Flower
Anugraham	Blessing	*Raajyam*	Country
Ariyikkuka	To inform	*Samudram*	Ocean
Baalan	Boy	*Satyam*	Truth
Bhoomi	Earth	*Shivan*	God Siva in Hindu mythology
Brahmaavu	God Brahma in Hindu mythology	*Simham*	Lion
Chandran	Moon	*Sooryan*	Sun
Chundu	Lips	*Svargam*	Heaven
Gunam	Goodness	*Thaamara*	Lotus
Irikkuka	To sit	*Thalamudi*	Hair
Kaadu	Forest	*Thaliru*	Newly sprouted leaves
Kallan	Thief	*Urakkam*	Sleep
Kannu	Eyes	*Veedu*	House
Kashtam	Difficult	*Vishnu*	God Vishnu in Hindu mythology
Kuyil	Cuckoo (A bird)	*Vriksham*	Tree

Aakasham_1_081 to *Aakasham_1_112* and *Aakasham_1_173* to *Aakasham_1_199* are extracted. These manually extracted keyframes are used as the ground truth values for our automatic keyframe extraction model. Some of the manually identified keyframes of the video *Aakasham_1* are shown in Figure 4.

Fig. 4. Examples of manually identified keyframes for video *Aakasham_1*. (a) corresponds to class *Aakasham_A*, (b) corresponds to class *Aakasham_B*, and (c) corresponds to class *Aakasham_C*

Each of these three sets from each word was considered as separate classes - A, B or C - with the corresponding word and was named in the format <Word>_<A, B or C>. For example, from the video *Aakasham_1*, we get frame numbers 1-24, 81-112, and 173-199 as three sets of keyframes. Therefore, the frames *Aakasham_1_001* to *Aakasham_1_024* belongs to class *Aakasham_A*, *Aakasham_1_081* to *Aakasham_1_112* belongs to class *Aakasham_B*, and *Aakasham_1_173*

to *Aakasham_1_199* belongs to class *Aakasham_C*. Similarly, in the case of video *Aakasham_2*, we get frame numbers 1-9, 158-185, and 321-346 as three sets of keyframes. They also belong to classes *Aakasham_A*, *Aakasham_B* and *Aakasham_C* respectively. Figure 4 depicts the three classes, with the first image from the left corresponding to class *Aakasham_A*, the next image corresponding to class *Aakasham_B* and the last image corresponding to class *Aakasham_C*. Repeating this process across every video for every word allowed us to create 123 classes from 41 words and populate the classes with manually identified keyframes corresponding to each class.

3.2.2 Keyframe Extractor [1]. We used a combination of CNN models with the CLIP Vision Language Model [43] and Bi-LSTM [18] on forty-one words, and classified the frames into keyframes and transition frames. The CNN Models used are ResNet50 [20], DenseNet121[24], EfficientNetB0 [54], and MobileNetV2 [47]. The features identified by each of the CNN models are concatenated with the features identified by the CLIP model. Since our videos consist of both general features such as shapes and edges, as well as Kathakali-specific semantic features such as *Mudras* and *Navarasas*, using only a CNN model or CLIP model will prove insufficient. Therefore, we decided to use a combination of the two for feature extraction. A Bi-LSTM was used to identify the temporal dependencies between the concatenated features. The models give a probability for every frame taken, and we selected all the keyframes predicted as output with a probability greater than or equal to 0.5. Each of the models gave us an accuracy of 76-77%. We selected the MobileNetV2+CLIP+Bi-LSTM model to extract the keyframes since this was the most lightweight model out of the various combinations experimented with. The probability threshold of 0.5 was selected after experimenting with different values. A threshold of 0.3 gave us too many frames, most of them being transition frames, and therefore was rejected. A threshold of 0.7 was too strict, leading to not enough keyframes being generated, and hence was rejected. From the 41 words with 10 videos each, we got a total of 6262 keyframes across 123 classes. Some of the keyframes predicted for the video *Aakasham_1* are given in Figure 5. The distribution of images for each word and for every class is shown in Table 6 in Appendix B.

Fig. 5. Examples of Predicted Keyframes for video *Aakasham_1*. (a) corresponds to class *Aakasham_A*, (b) corresponds to class *Aakasham_B*, and (c) corresponds to class *Aakasham_C*

From Table 6, we can see that the word *Aakasham* contains 120 images, out of which 49 images belong to the class *Aakasham_A*, 35 images to class *Aakasham_B* and 36 to class *Aakasham_C*. We can see that our dataset is highly imbalanced, with classes *Nee_Shringaaram_B* and *Chandran_C* containing only 6 and 9 keyframes, respectively, while the class *Thaliru_C* contains 159 keyframes. To address the data imbalance problem, we oversampled the keyframes

[1]This work has been submitted to a conference

to correspond to the class with the maximum number of keyframes (*Thaliru_C* with 159 keyframes). However, this issue led to severe overfitting and data leakage across training, validation and test splits, with copies of the same image being present in training and testing. For this reason, we decided not to perform oversampling and instead decided to use the keyframes as they were.

3.3 Pose estimation and Normalisation

3.3.1 Pose estimation. After the extraction of keyframes, our next task was to perform pose estimation and normalisation on the keyframes. Pose estimation is the process of detecting keypoints from images [60]. Since our videos were recorded with a single artist, our video dataset could be interpreted as too specific and skewed. To remove this problem and to make our approach more generalised, we decided to create skeletal postures for all the frames and use them in our model. This way, any person could be represented using skeletal images and the problem of the model learning useless information, such as skin colour, background, etc., is removed. We decided to use DWPose pose estimation [59] for our purposes, since it gives better representation across fingers and can handle occlusion better than other pose estimation models, like MediaPipe and OpenPose. DWPose gives us a list of keypoints pertaining to parts of the body, and the skeletal images are plotted on a black background. We decided to consider only the keypoints with respect to the neck, nose, shoulders, arms and fingers. This decision was based on the fact that head and lower body movements do not contribute towards the semantic representation of the words, it rather contributes towards the dance and dramatic representation of the words. Whereas the neck, shoulders, arms and fingers contribute towards the semantics of the word being represented. Consequently, we removed the lower portion of the body, including everything from the waist. Even though facial expressions are important in Kathakali, for our work, we decided to focus only on the hands and upper body postures. As a consequence, we removed the facial keypoints as well.

3.3.2 Normalisation. In order to generalise the pose estimated keyframes, we applied normalisation techniques, namely translation and scaling transforms.

The translation transform ensures that the person in the skeletal image is always in the same place, regardless of where they are in the original image. We fixed the neck at the location (0,0) in the X-Y plane of the skeletal image, which falls at the exact centre of our image. The equation for translation is given below -

$$\begin{aligned} x' &= x + t_x \\ y' &= y + t_y \end{aligned} \tag{1}$$

where x and y are the original coordinates, x' and y' are the translated coordinates and t_x and t_y are the distance along x and y axes. For example, if the original coordinates of the neck were at (300, 400), and its new coordinates are (0,0), then t_x and t_y would be -300 and -400, respectively. We do the same process for every keypoint in order to translate the other keypoints as well, near the neck in the translated image.

Our next step is to scale the limbs by the shoulder keypoints to obtain standardisation for the skeletal length. We first computed the distance between the shoulder keypoints (shoulder width) and then computed the angles between every pair of joints. We scaled the shoulder width to 1. Further, we scaled the remaining limbs in the pose-estimated image after translation by dividing the limb length by the shoulder width. New keypoints are constructed using their corresponding normalised limb lengths and the computed angle between joints. The equation used for identifying the new keypoints after scaling is given below -

$$x'' = x' + r\cos\theta$$
$$y'' = y' + r\sin\theta \tag{2}$$

where x'' and y'' are the new coordinates, x' and y' are the translated coordinates from the previous step, r is the normalized limb length, and θ is the angle between the limbs

The skeletal images are created with a size of 2300x2160 pixels to ensure that the newly created skeletal images do not go out of the frame. Some of the created skeletal keyframes for Aakasham are shown in Figure 6.

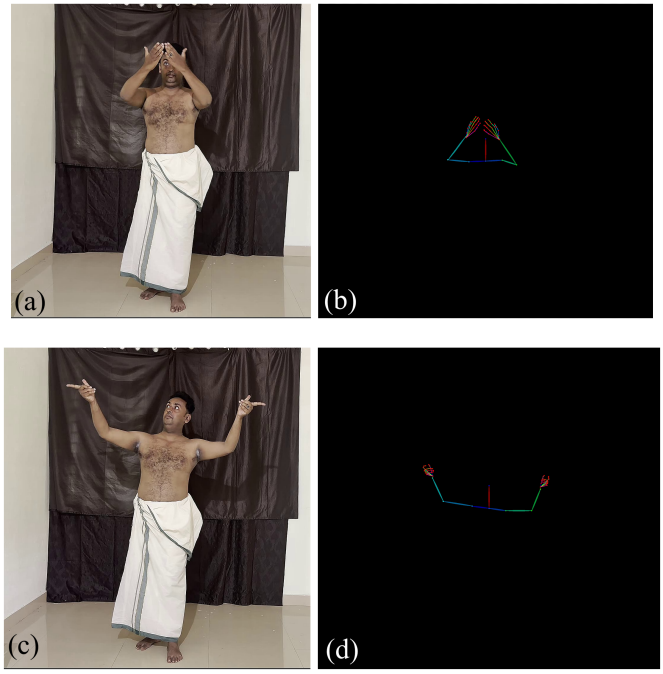

Fig. 6. Examples of Skeletal keyframes for video *Aakasham_1* after normalization. (a) is the original automatically identified keyframe from class *Aakasham_A*, (b) is it's normalized skeletal equivalent, (c) is the original automatically identified keyframe from class *Aakasham_C*, and (d) is it's normalized skeletal equivalent

3.4 Feature Extraction and Correlation Analysis

After keyframe extraction, pose estimation, and normalisation, we proceeded to extract features from the images. There are 50 normalised keypoints in the normalised skeletal images, each keypoint consisting of x and y coordinates. Each of the 50 keypoints is split into its corresponding x and y coordinates, thereby totalling 100 features. Further, we also decided to engineer features from the images and keypoints, namely the body angles and finger-to-wrist distances. The upper body angles include 44 across every joint, including the angles between every finger. Corresponding to each of the 10 fingers, the finger-to-wrist distances are computed using Euclidean distance from the keypoints of each fingertip to the keypoints of the wrist. After extracting these 154 pose features, we decided to run correlation analysis on these features to ensure that redundant features were removed. The Correlation Heatmap is shown in Figure 7.

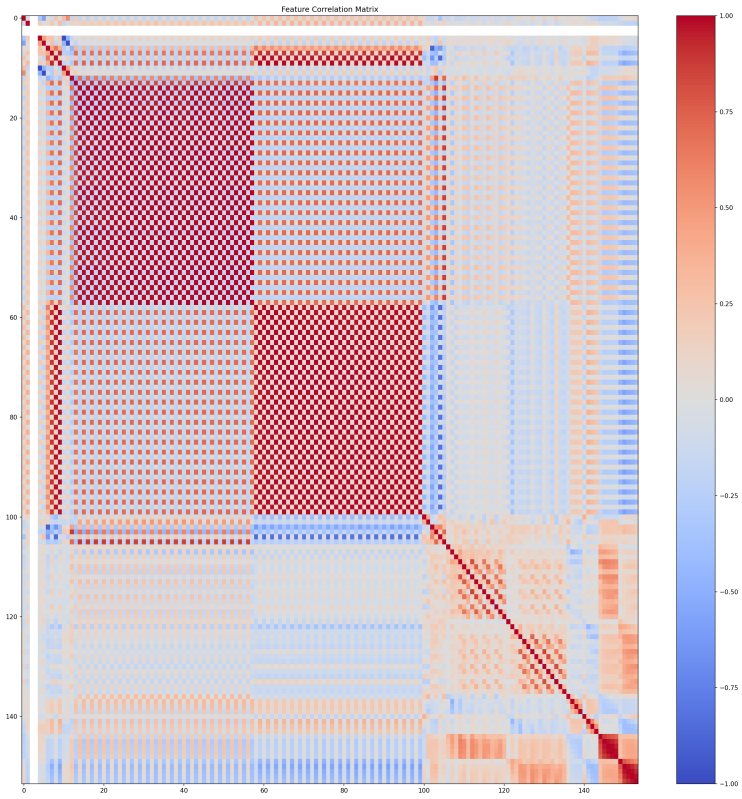

Fig. 7. Correlation Heatmap

From the heatmap, we can infer that a lot of features are correlated with each other. The first 12 features, corresponding to the x and y coordinates of the neck, nose, shoulders and elbows, are not significantly correlated with each other. The next 44 features, corresponding to the left hand finger keypoints, showed that the x coordinates of different keypoints of the left hand are positively correlated with each other and are negatively correlated with the y coordinates. The y coordinates also follow the same pattern, being positively correlated with y coordinates and negatively correlated with x coordinates. The next 44 features, corresponding to the coordinates on the right hand, also followed the same trend.

The next 6 features correspond to the body angles between left and right shoulders, elbows and wrists. The next 15 features correspond to the right hand finger joint angles, and the next 15 correspond to the left hand finger joint angles. 8 features correspond to the angles between fingers in the right and left hands, respectively. Across these 44 angles, we do not see a high value in correlation between them.

The last 10 features correspond to the finger-to-wrist distances, 5 for the right hand and 5 for the left hand. These are correlated with each other, though not as low as the correlation of the angles, and not as high as the correlation between keypoints.

Since we realised that most of the keypoint coordinates are correlated to each other, we decided to remove those. After experimentation, we removed features with correlation above 0.98. This helped us remove all the keypoints of the

fingers, but retain keypoints of the nose, shoulders, etc. It also included two finger-to-wrist distances, corresponding to the distance between the left thumb and left wrist and the right thumb and right wrist. This brought down our total number of pose features to 66.

3.5 Model Architecture and Image Classification

We create a novel fusion model for image classification. To ensure both images and the features are trained, we built two separate modules, one corresponding to images and the other corresponding to the pose features and then finally concatenated them in a third module. The detailed architecture diagram is shown in Figure 8.

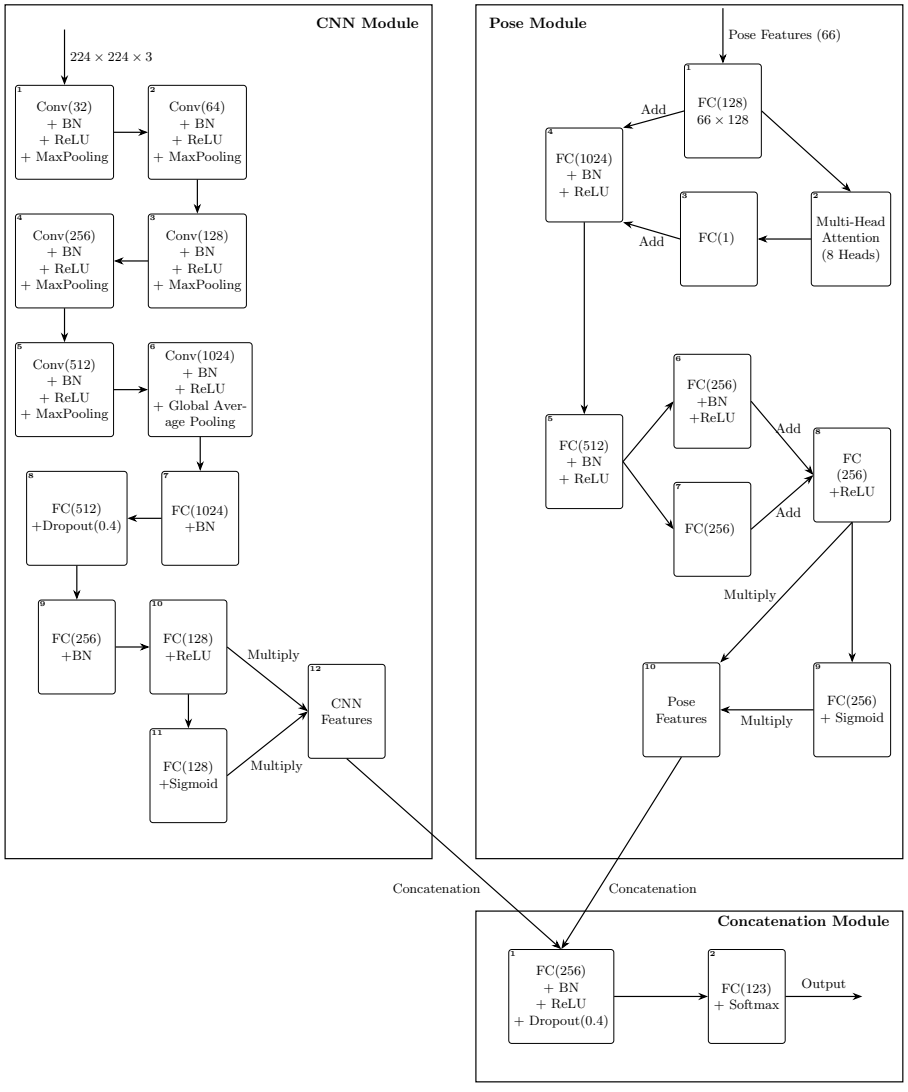

Fig. 8. Proposed architecture for image classification

The first module consists of CNN layers that process the images, which are converted to a size of 224x224x3 (the most common image size used across existing CNN models). The second module processes the pose features with multi-head attention and fully connected layers. The final module concatenates the output from the previous two modules and predicts one of the 123 classes as output.

3.5.1 CNN Module. The CNN module consists of twelve blocks, each numbered from one to twelve, where hierarchical feature learning consisting of expansion, bottleneck and compression is performed, which is finally followed by feature attention. The convolution layers are expanded in size from 32 to 1024 with batch normalisation, ReLU, and pooling layers (blocks 1 to 6), followed by fully connected layers which compress the size from 1024 to 128 progressively, with dropout, batch normalisation, and ReLU added across the layers (blocks 7 to 10). The expansion allows the model to learn low-level features in the initial blocks and high-level features in the later blocks. The compression that follows the expansion allows the model to compress the high-level representations into a more compact form. This expansion and compression across the layers help to retain only the most important features needed for accurate classification and allow the model to concentrate more on those features. Finally, feature attention is calculated by performing element-wise multiplication using the Sigmoid activation function (blocks 10 and 11) to get the final CNN features. This feature attention will help the model understand which features are useful among the most important features identified after the expansion and compression.

3.5.2 Pose Module. The Pose module consists of ten blocks, each numbered from one to ten. The sixty-six pose features taken as input are converted to 128-dimensional vectors, and a multi-head attention [56] with 8 heads is added (blocks 1 to 3). This multi-head attention helps to understand which features are important relative to each other and helps to concentrate more on the important features. This is followed by the addition of the original features, an idea inspired by skip connections in the ResNet architecture [20]. This helps to ensure that the original pose features are still present if the attention block loses any useful information. These are progressively converted to sizes 1024 to 256 across the next few layers with batch normalisation and ReLU (blocks 4 to 6). The projection from size 66 to 1024 would allow the network to learn many interaction patterns simultaneously by projecting the features to a higher dimension. A second addition follows this (block 8) to retain the original pose features. Finally, feature attention is calculated by performing element-wise multiplication using the Sigmoid activation function (blocks 8 and 9) to get the final Pose features. This feature attention is similar to the feature attention in the CNN module and is used to identify which features are useful.

3.5.3 Concatenation Module. Finally, in the third module, the CNN features and the Pose features (outputs of the CNN and Pose blocks) are concatenated together, followed by fully connected layers, batch normalisation, ReLU activation function and dropout. The last layer has a size of 123, corresponding to the number of classes we have, with a Softmax activation function [9] to get the final output.

3.6 Word Prediction from the videos

Our next task is to predict the word corresponding to the input video. Based on the keyframe classification, we select the keyframes that were classified to a class with a probability greater than or equal to 0.5. From the selected keyframes, we predict the word by selecting *Word* to which the maximum number of keyframes belong. Note that here the *Word* refers to the first part of the class name.

For example, consider eight keyframes identified from a video, and they are identified as belonging to classes *Aakasham_A*, *Aakasham_A*, *Aakasham_B*, *Svargam_B*, *Veedu_A*, *Aakasham_B*, *Aakasham_C* and *Aakasham_C*. Consider their Softmax probabilities to be 0.69, 0.42, 0.87, 0.92, 0.67, 0.89, 0.91, and 0.23, respectively. After removing the frames with Softmax probability less than 0.5, the remaining frames are the ones identified as belonging to classes *Aakasham_A*, *Aakasham_B*, *Svargam_B*, *Veedu_A*, *Aakasham_B*, and *Aakasham_C*. We see that the maximum number of frames belong to *Aakasham*. Therefore, we conclude that the word associated with the video is Aakasham.

In case of ties between two or more words being predicted as the majority, we find the normalised Softmax probability of the majority words as a tie-breaker. We again take all the frames which have a Softmax probability greater than or equal to 0.5. For the majority words identified in a video, the normalisation is performed by summing the probability of each keyframe of that *Word* and dividing it by the total number of keyframes of that *Word*. The majority word with the highest normalised probability is predicted as the output. In case there is a tie again in the normalised probabilities, we output all the majority words.

For example, five keyframes are identified from a video after filtering with Softmax probability greater than 0.5, and they are identified as *Vriksham_B*, *Vriksham_C*, *Kaadu_C*, *Kaadu_C*, and *Aana_A*. Consider that the Softmax probabilities are 0.54, 0.72, 0.71, 0.66, and 0.98, respectively. A tie exists between *Vriksham* and *Kaadu*, making them the majority words. We find the normalised probability of *Vriksham* by summing the Softmax probabilities of all the frames identified as *Vriksham* and dividing by the number of keyframes identified as *Vriksham*. Therefore, the normalised probability of *Vriksham* becomes 0.63 (the sum of 0.54 and 0.72 is divided by 2). Similarly, the normalised probability of *Kaadu* becomes 0.69. Since the normalised probability of *Kaadu* is higher than that of *Vriksham*, the word is identified as *Kaadu*.

4 Results and Discussions

All the experiments were performed on a 20GB partition of an Nvidia H100 80 GB GPU. The experiments were run using Python 3.13.3. We used classification metrics such as Accuracy, Precision, Recall, and F1-score to evaluate the effectiveness of our work. To ensure class imbalance is handled, weighted versions of precision, recall, and F1-score are the metrics used.

4.1 Image Classification

To effectively perform image classification, we decided to split the videos. We split the videos into a 70:10:20 train-validation-test split such that 7 videos of each word are used for training, 1 video for validation and 2 videos for testing. A total of 6262 keyframes are generated from the dataset (410 videos), out of which 4441 keyframes (from 287 videos) are used for training, and 631 keyframes (from 41 videos) are used for validation. The best-performing model was saved and was used on the 1190 test images (from 82 videos).

Our image classification model was trained for 50 epochs. We used a batch size of 32 with Adam optimiser, an initial learning rate of 0.0001, a CosineDecay scheduler [37] to change the learning rate dynamically across the epochs, and Categorical Cross Entropy with label smoothing of 0.1 to calculate the error.

To compare the performance of our model with other models, we also trained a few other benchmark CNN models such as MobileNetV2 [47], VGG19 [52], ResNet50 [20], EfficientNetB0 [54], and DenseNet121 [24] on our dataset. To enable these benchmark models to train on our data, we replaced our CNN module with the existing pre-trained CNN models and kept the Pose and Concatenation modules as they were. We also decided to see how these models would

Table 2. Comparison of image classification using our model with existing image classification models

Model	Accuracy	Precision	Recall	F1-Score
MobileNetV2 (without pose + Attention)	0.6773	0.69	0.68	0.66
MobileNetV2 (with pose + attention)	0.7983	0.73	0.75	0.73
VGG19 (without pose + attention)	0.7008	0.72	0.70	0.68
VGG19 (with pose + attention)	0.7798	0.71	0.72	0.70
ResNet50 (without pose + attention)	0.6739	0.60	0.61	0.57
ResNet50 (with pose + attention)	0.7773	0.71	0.73	0.70
EfficientNetB0 (without pose + attention)	0.6622	0.60	0.61	0.57
EfficientNetB0 (with pose + attention)	0.7731	0.71	0.72	0.69
DenseNet121 (without pose + attention)	0.6319	0.56	0.57	0.53
DenseNet121 (with pose + attention)	0.7857	0.72	0.73	0.70
Proposed image classification model (without pose + attention)	**0.7866**	**0.74**	**0.74**	**0.71**
Proposed image classification model	**0.8277**	**0.78**	**0.80**	**0.77**

run without the pose and concatenation modules, and to ensure consistency, we also did the same to our model. These results are shown in Table 2.

From the table, we can see that the addition of pose features and attention improves the performance of each of the models. The lower values of the metrics from the models without pose features and attention compared to the ones with pose and attention tell us that image features are not sufficient for the classification. Different classes may have similar body angles and hand postures, and using the image features may not prove to be sufficient in these cases. We can therefore observe that the addition of pose and attention definitely improves the accuracy, and this improvement is not specific to one CNN and is visible in all the architectures.

The proposed architecture has an accuracy of 78.66% without pose and attention, which is higher than that of the benchmark models. With the addition of pose and attention, the accuracy increases by around 4%. There is an increase in the values of the other metrics as well, especially with recall jumping from 74 to 80% with the addition of pose features and attention. This increase suggests that the model is able to better identify the classes each image belongs to, therefore having a higher value in True Positives, and thereby increasing the Recall.

4.1.1 Experiments with architecture and pipeline. While creating the model, we experimented with different fully connected layer sizes, filter sizes, number of filters in convolution layers, etc. The results of these experiments are given in Table 7 in Appendix C.1. From this table, it is clear to understand that the architectural design choices behind our model and it proves that the current model performs better compared to the rest of the experiments. These are summarised below -

- All the experimental variants result in accuracies below our proposed model, though the best one of those, the model with attention dimension 64 and 4 heads, is a suitable alternative in low-resource situations.
- The experiments involving increasing or decreasing the dimensions of the intermediate layers in the modules, or the correlation thresholds, clearly show the reason behind the various values selected in the pipeline as well as the model.

Table 3. Results of Ablation studies

Model	Accuracy	Precision	Recall	F1-Score
Without attention	0.8076	0.77	0.77	0.74
Without both the Multiply layers	0.7950	0.75	0.76	0.72
Without the CNN Multiply layer	0.8092	0.75	0.75	0.72
Without the Pose Multiply layer	0.8042	0.75	0.75	0.72
CNN module + Concatenation module (No pose module)	0.7866	0.74	0.74	0.71
Without last CNN layer	0.7975	0.75	0.75	0.72
Without last 2 CNN layers	0.7992	0.74	0.76	0.72
Pose module + Concatenation module (No CNN module)	0.3882	0.32	0.32	0.29
Without both the Addition layers	0.7697	0.73	0.72	0.69
Without first addition layer	0.7756	0.72	0.74	0.70
Without second addition layer	0.8025	0.76	0.77	0.74

- We can see that colour also provides information to the images, with different limbs represented by different colours. This allows the model to learn different limbs and helps it to classify better, shown by the decrease in accuracy when using greyscale images.
- Usage of a 5x5 filter size also does not result in improvement in our model; this could be due to unnecessary information being learnt by the model in the earlier layers, such as parts of the black background.
- The proposed model also has higher values of precision, recall, and F1-score, whereas most of the experiments have lower values comparatively. This shows that the proposed model's over other classification metrics is also high, and is not just limited to accuracy.

4.1.2 Experiments with feature combinations. We also experimented with changing the extracted and engineered features given as input to the Pose module. We experimented with only using coordinates, angles, and finger-to-wrist distances, as well as with every combination of any two of the above three. The results of these experiments are given in Table 8 in Appendix C.2.

From the results, we can see that each of the features provides useful information to the model. Among the individual features, angles perform best, showing that the model is able to learn the approximate angles between the features and use this for classification. This is also reinforced by the fact that the angles are the features with the highest number, with forty-four of the sixty-six features being angles. When it comes to the experiments with feature combinations, we can see that combining features improves the accuracy. Interestingly, the combination of coordinates and finger-to-wrist distances performs better than the experiments with angles in combinations. Though the combinations of coordinates and finger-to-wrist distances comprise only twenty-two features in total, the coordinates provide the spatial locations of the hands, and the finger-to-wrist distances are able to describe the shape and size of the hands, thereby indirectly helping to identify different *Mudras*. However, our proposed model, which combines all three features, provides the best performance with the highest accuracy. This is visible in the case of the other metrics as well, with our model achieving the highest precision, recall, and F1-score.

4.2 Ablation Studies on Image Classification Architecture

In order to find out what the performance of the model would be if certain components were removed from it, we ran ablation studies with our model. The results are summarised in Table 3 and are summarised below.

- Removing the attention block fully led to a decrease in the metric values, indicating that the attention mechanism contributes to the model's ability to learn the importance of different pose features relative to each other.
- Next, we tried to see how the performance would be if we removed the multiplications (feature importances). We tried removing one as well as both multiplications, leading to a decrease in the metrics. This shows us that these feature attentions in both the CNN and pose branches give necessary information to the model by suggesting which features are more important, and thereby help the model focus more on them.
- We then tried to remove either the CNN or the Pose module and ran the image classification with the remaining modules, with results showing a drop in metrics upon removal (The removal of pose features and attention was already shown in Table 2). There is a larger degradation in model performance when the CNN module is removed, which suggests that the images given as input provide more information to the model as compared to the pose module. However, the usage of the pose module is also necessary, as shown by the drop in metrics when only the CNN module was used.
- Further, on our original model, we also tried experimenting by removing CNN blocks. Since the last two CNN blocks are comparatively heavier, removal of these blocks would make the architecture lighter. We experimented by removing both, as well as the heavier one, with both leading to a drop in metrics. This shows us that even if the model architecture size becomes lighter, the projection to higher dimensions helps the model be more accurate.
- Finally, we tried to evaluate the importance of additions by removing one as well as both of them, with all cases leading to a reduction in metric values. The smaller decrease when only the second addition is removed suggests that the first one may have a greater contribution in the architecture. However, since the skip connections were introduced to mitigate the vanishing gradient problem, their removal could lead to gradient values not being updated, in turn leading to erroneous outputs, as evidenced by the reduction in metrics.

4.3 Video Classification

Next, we performed video classification on our 82 test videos. We followed the full pipeline: automatic keyframe extraction, pose estimation and normalisation, image classification on the keyframes, and finally video classification using majority voting and normalised probabilities. To compare the performance of our model, we also performed video classification on our videos using existing models. The models we selected were LSTM [22] and Transformer models.

To extract features before using LSTM, we used benchmark CNN models like ResNet50 [20], DenseNet121 [24], EfficientNetB0 [54], MobileNetV2 [47] and VGG19 [52]. These CNN models extracted the inputs, passed to an LSTM layer of size 512, and then passed through a fully connected layer of size 256 and finally another fully connected layer with size 41, corresponding to the number of classes (number of words we have).

In the case of transformer models, we decided to go with benchmark transformer models for image data, which were Vision Transformer (ViT) [16] and SWIN Transformer [36] models. The transformer models took the input and performed multi-head attention with 8 heads, then passed through a fully connected layer of size 256 and finally another fully connected layer with size 41, corresponding to the number of classes.

The train-validation-test split was fixed at 70:10:20, such that for each word, 7 videos were used for training, 1 video was used for validation, and 2 videos were used for testing. We fixed the batch size at 32, and the models were run for

Table 4. Results of Video classification

Model	Accuracy	Precision	Recall	F1-Score
MobileNetv2 + LSTM	0.8780	0.86	0.88	0.85
EfficientNetB0 + LSTM	0.9268	0.93	0.93	0.92
DenseNet121 + LSTM	0.8780	0.85	0.88	0.85
ResNet50 + LSTM	0.8659	0.86	0.87	0.85
VGG19 + LSTM	0.8293	0.82	0.83	0.80
ViT Transformer	0.9390	0.96	0.94	0.93
SWIN Transformer	0.8659	0.86	0.87	0.84
Our pipeline	**0.9756**	**0.98**	**0.98**	**0.97**

50 epochs. This ensured that the comparison models followed the same settings as our image classification model. The results of the comparison are given in Table 4.

From the results, we can see that our pipeline leads with higher metric values than those of the existing models. It is interesting to note that, with the exception of EfficientNetB0 + LSTM, the other CNN + LSTM combinations have reduced accuracy. We can also note that the EfficientNetB0 + LSTM combination performs better than the SWIN transformer. This tells us that the features identified by each CNN model are different. The usage of hierarchical feature representation learnt by EfficientNetB0, as compared to the depth-wise separate convolutions used by MobileNetV2, heavy skip connections used by ResNet50, dense feature reuse by DenseNet121, and the convolutional architecture used by VGG19. We can observe that the addition of LSTM does not compensate for the features identified by the CNNs. ViT uses global self-attention across images, whereas SWIN performs attention through local windows and connects them through shifted-window operations. Since word identification depends on different relationships between body angles and hands, which may be possible using ViT but not with SWIN. These results show us that the accuracy of the classification depends more on how each model utilises the visual information being fed to it, rather than the model size.

To test the generalizability of our pipeline, we took one video each using four different people for each of the forty-one words. We then tested these videos on our pipeline and saw that 70% of these videos were predicted correctly.

4.4 Proposed Algorithm

In this section, we propose an algorithm based on our full pipeline to identify a word signed in an input video, described using Algorithm 1 given below.

5 Conclusion

The work presents a pipeline to enable video-to-word translation of Kathakali dance postures using a novel image classification model. We first recorded dance movements with a single Kathakali artist for forty-one words. We developed a model using a combination of CNN, CLIP and Bi-LSTM to automatically detect the keyframes from the videos. We converted these to pose-estimated normalised skeletal keyframes, extracted 66 pose features, and classified the keyframes using a novel deep learning image classification model we developed. Further, video classification was done on the keyframes from a video using majority voting, and normalised probability scores were used as a tie-breaker. Finally, we present an algorithm detailing our approach for video-to-word translation.

Algorithm 1 Automatic Video Classification

Input: Input video V
Output: Predicted word W
1: Extract all the frames from V
2: Select keyframes using the keyframe extraction model
   (a) Perform pose estimation for each keyframe using DWPose
   (b) Normalise the skeletal keyframe
   (c) Extract 66 pose features from each keyframe
   (d) Classify each keyframe using the classification model, which takes as input the normalised skeletal keyframes and pose features
3: Perform majority voting and normalised probability to identify the word from the classification output of keyframes
4: **return** W

A few limitations exist in this work. A main limitation is that our dataset is small, consisting of only forty-one words with ten videos each. It is also skewed, with only one person acting out all forty-one words. Another limitation is that each word uses three classes, corresponding to the number of key positions in the video. Scaling the dataset size to a huge value like a 1000 words could lead to significant issues, with a 1000 words leading to a 3000-class classification, which needs very high computing resources. Further, increasing the number of classes would require a model with more CNN layers, features, and larger attention parameters to get better accuracy, which in turn could lead to mathematical resource constraints as well.

In future, we plan to collect more words, with different people, to ensure our dataset emulates real-life data. This could help us publish this dataset as a Benchmark dataset for Kathakali word prediction. We also plan to pursue other methods, such as ontological modelling techniques, for video classification, which would work well when the dataset is scaled and at the same time does not lose accuracy. This novel pipeline can help us preserve the intangible heritage of India through the digital preservation of ancient art forms such as Kathakali, by digitally storing and predicting the words the postures represent. In future, this methodology can be adopted for various classical dance forms across the Indian subcontinent.

References

[1] S Asad Ahmed, Y Jabir Sheriff, B Prakash, and Suresh Babu Thandullu Naganathan. 2024. Sign language to text translator: A semantic approach with ontological framework. In *2024 10th International Conference on Communication and Signal Processing (ICCSP)*. IEEE, 736–741.

[2] Giuseppe Amato, Fabrizio Falchi, and Claudio Gennaro. 2015. Fast Image Classification for Monument Recognition. *J. Comput. Cult. Herit.* 8, 4, Article 18 (Aug. 2015), 25 pages. doi:10.1145/2724727

[3] Tejaswini Ananthanarayana, Priyanshu Srivastava, Akash Chintha, Akhil Santha, Brian Landy, Joseph Panaro, Andre Webster, Nikunj Kotecha, Shagan Sah, Thomastine Sarchet, et al. 2021. Deep learning methods for sign language translation. *ACM Transactions on Accessible Computing (TACCESS)* 14, 4 (2021), 1–30.

[4] Anonymous. 1892. *Hasta Lakshana Deepika.* Janranjinee Achukoodam, Nadapuram, Kerala.

[5] Andrea Asperti, Leonardo Dessì, and Nico Wu. 2026. Art through CLIP's Eyes. *J. Comput. Cult. Herit.* 19, 2, Article 32 (June 2026), 33 pages. doi:10.1145/3812546

[6] Nikoloas Bakalos, Eftychios Protopapadakis, Anastasios Doulamis, and Nikolaos Doulamis. 2018. Dance posture/steps classification using 3D joints from the kinect sensors. In *2018 IEEE 16th Intl Conf on Dependable, Autonomic and Secure Computing, 16th Intl Conf on Pervasive Intelligence and Computing, 4th Intl Conf on Big Data Intelligence and Computing and Cyber Science and Technology Congress (DASC/PiCom/DataCom/CyberSciTech)*. IEEE, 868–873.

[7] Himadri Bhuyan, Partha Pratim Das, Jatindra Kumar Dash, and Jagadeesh Killi. 2021. An automated method for identification of key frames in Bharatanatyam dance videos. *IEEE Access* 9 (2021), 72670–72680.

[8] Zaid Saad Bilal, Amir Gargouri, Hanaa F Mahmood, and Hassene Mnif. 2024. Advancements in Arabic Sign Language Recognition: A Method based on Deep Learning to Improve Communication Access. *J. Internet Serv. Inf. Secur.* 14, 4 (2024), 278–291.

[9] John Bridle. 1989. Training stochastic model recognition algorithms as networks can lead to maximum mutual information estimation of parameters. *Advances in neural information processing systems* 2 (1989).

[10] Antoine Brimont, Titus Zaharia, and Ruxandra Tapu. 2026. A Survey on Video Captioning in the Era of Large Language Models. *ACM Transactions on Multimedia Computing, Communications and Applications* (2026).

[11] Pranav Chaudhari, Alex Vicente-Sola, Amlan Basu, Davide L Manna, Paul Kirkland, and Gaetano Di Caterina. 2024. Sign language recognition using spiking neural networks. *Procedia Computer Science* 235 (2024), 2674–2683.

[12] Jian-Feng Chen, Hong Lu, Renzhong Wei, Cheng Jin, and Xiangyang Xue. 2010. An effective method for video genre classification. In *Proceedings of the ACM International Conference on Image and Video Retrieval*. IEEE, 97–104. doi:10.1145/1816041.1816058

[13] Phuc Dang Thi, Hoang Thi Nguyen Dang, Phuc Doan Huu, and Hieu Dau Sy. 2025. Video classification for efficient data storage using deep learning: a comparison of sequential and simultaneous feature extraction methods. *Multimedia Tools and Applications* 84, 6 (2025), 3071–3094.

[14] Marcello Di Giammarco, Camilla Vitulli, Simone Cirnelli, Benedetta Masone, Antonella Santone, Mario Cesarelli, Fabio Martinelli, and Francesco Mercaldo. 2025. Explainable deep learning for breast cancer classification and localization. *ACM Transactions on Computing for Healthcare* 6, 1 (2025), 1–18.

[15] Swapna Donepudi, KN Divya, Reemasen Tungala, Sirisha Thammisetty, et al. 2025. A Sign Language Recognition System for Understanding Actions of Speech-Impaired-Individuals. In *2025 7th International Conference on Intelligent Sustainable Systems (ICISS)*. IEEE, 1303–1310.

[16] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. 2020. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929* (2020).

[17] Eman K Elsayed and Doaa R Fathy. 2020. Sign language semantic translation system using ontology and deep learning. *International Journal of Advanced Computer Science and Applications* 11, 1 (2020).

[18] Alex Graves, Santiago Fernández, and Jürgen Schmidhuber. 2005. Bidirectional LSTM networks for improved phoneme classification and recognition. In *International conference on artificial neural networks*. Springer, 799–804.

[19] Sharish Gupta and Sarbjeet Singh. 2024. Indian dance classification using machine learning techniques: A survey. *Entertainment Computing* 50 (2024), 100639.

[20] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 770–778.

[21] Gines Hidalgo, Yaser Sheikh, K Kitani, Aayush Bansal, Ramon Sanabria, Donglai Xiang, Xiu Li, and Haroon Idrees. 2019. OpenPose: Whole-body pose estimation.

[22] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long short-term memory. *Neural computation* 9, 8 (1997), 1735–1780.

[23] Zhen Hong, Miao Hong, Ning Wang, Yong Ma, Xiaolong Zhou, and Wei Wang. 2022. A wearable-based posture recognition system with AI-assisted approach for healthcare IoT. *Future Generation Computer Systems* 127 (2022), 286–296.

[24] Gao Huang, Zhuang Liu, Laurens Van Der Maaten, and Kilian Q Weinberger. 2017. Densely connected convolutional networks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 4700–4708.

[25] Shilpa Ingoley and Jagdish Bakal. 2025. Interpretation of Indian sign language to text and speech to communicate with speech and hearing impaired community. *Procedia Computer Science* 258 (2025), 1980–1992.

[26] Ganesh Neelakanta Iyer, Nattapong Saengarunvong, and Minh Hien Le. 2024. When artificial intelligence meets navarasas-facial expression classification in kathakali using dsfd and cnn. In *2024 IEEE International Conference on Signal Processing, Informatics, Communication and Energy Systems (SPICES)*. IEEE, 1–6.

[27] Hamid Reza Vaezi Joze, Amirreza Shaban, Michael L Iuzzolino, and Kazuhito Koishida. 2020. MMTM: Multimodal transfer module for CNN fusion. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 13289–13299.

[28] S Kanimozhi, A Sasithradevi, and L Sairamesh. 2024. Ontology-Based Action Recognition in Sport Videos Using Semantic Verification Model. *IEEE Access* 12 (2024), 96783–96796.

[29] Shagil Kannur. 2024. *Kathakali of Kerala at Nishagandhi dance festival 2024 (197).* https://commons.wikimedia.org/wiki/File:Kathakali_of_Kerala_at_Nishagandhi_dance_festival_2024_(197).jpg

[30] Huizhu Kao. 2024. Multi-person dance tiered posture recognition with cross progressive multi-resolution representation integration. *PloS one* 19, 6 (2024), e0300837.

[31] Pakhee Kumar, Ferda Ofli, Muhammad Imran, and Carlos Castillo. 2020. Detection of Disaster-Affected Cultural Heritage Sites from Social Media Images Using Deep Learning Techniques. *J. Comput. Cult. Herit.* 13, 3, Article 23 (Aug. 2020). doi:10.1145/3383314

[32] Anas Laamouri and Sael Nawal. 2024. Image classification for visual recommendation using deep learning. In *Proceedings of the 7th International Conference on Networking, Intelligent Systems and Security*. Association for Computing Machinery, 1–6. doi:10.1145/3659677.3659693

[33] Dou Li. 2025. Computer Vision-Based Dynamic Capture of Intangible Cultural Heritage Crafts in New Media and Research on Digital Inheritance. In *Proceedings of the 2025 International Conference on Digital Society and Intelligent Computing*. 614–619.

[34] Hang Li, Hao Li, Ying Qin, and Yiming Liu. 2025. Optimization Method of Human Posture Recognition Based on Kinect V2 Sensor. *Biomimetics* 10, 4 (2025), 254.

[35] Ke Li, Shijie Wang, Xiang Zhang, Yifan Xu, Weijian Xu, and Zhuowen Tu. 2021. Pose recognition with cascade transformers. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*. 1944–1953.

[36] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. 2021. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*. 10012–10022.

[37] Ilya Loshchilov and Frank Hutter. 2016. Sgdr: Stochastic gradient descent with warm restarts. *arXiv preprint arXiv:1608.03983* (2016).

[38] Camillo Lugaresi, Jiuqiang Tang, Hadon Nash, Chris McClanahan, Esha Uboweja, Michael Hays, Fan Zhang, Chuo-Ling Chang, Ming Yong, Juhyun Lee, et al. 2019. Mediapipe: A framework for perceiving and processing reality. In *Third workshop on computer vision for AR/VR at IEEE computer vision and pattern recognition (CVPR)*, Vol. 2019. Long Beach, CA, 2.

[39] Yashashree Mahale, Nida Khan, Kunal Kulkarni, Shilpa Gite, Biswajeet Pradhan, Abdullah Alamri, Chang-Wook Lee, Mrinal Bachute, et al. 2025. A comprehensive approach for waste management with GAN-augmented classification. *PeerJ Computer Science* 11 (2025), e3156.

[40] Tanwi Mallick, Patha Pratim Das, and Arun Kumar Majumdar. 2021. Bharatanatyam dance transcription using multimedia ontology and machine learning. (2021), 179–222.

[41] Isaac Morales-Nolasco, Sandra Arias-Guzman, and Laura Garay-Jiménez. 2024. A method for complex posture recognition during long-term sitting using neural networks and pressure mapping systems. *Biomedical Signal Processing and Control* 95 (2024), 106306.

[42] Yulius Obi, Kent Samuel Claudio, Vetri Marvel Budiman, Said Achmad, and Aditya Kurniawan. 2023. Sign language recognition system for communicating to people with disabilities. *Procedia Computer Science* 216 (2023), 13–20.

[43] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. 2021. Learning transferable visual models from natural language supervision. In *International conference on machine learning*. PmLR, 8748–8763.

[44] S Renjith, Manazhy Rashmi, and Sumi Suresh. 2024. Sign language recognition by using spatio-temporal features. *Procedia Computer Science* 233 (2024), 353–362.

[45] Sriparna Saha, Anupam Banerjee, Sumana Basu, Amit Konar, and Atulya K Nagar. 2013. Fuzzy image matching for posture recognition in ballet dance. In *2013 IEEE international conference on fuzzy systems (FUZZ-IEEE)*. IEEE, 1–8.

[46] Soumitra Samanta, Pulak Purkait, and Bhabatosh Chanda. 2012. Indian Classical Dance classification by learning dance pose bases. In *2012 IEEE Workshop on the Applications of Computer Vision (WACV)*. IEEE, Breckenridge, CO, USA, 265–270. doi:10.1109/WACV.2012.6163050

[47] Mark Sandler, Andrew Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. 2018. Mobilenetv2: Inverted residuals and linear bottlenecks. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. 4510–4520.

[48] C Selvi, Y Anvitha, CH Asritha, and PB Sayannah. 2021. Kathakali face expression detection using deep learning techniques. In *Journal of Physics: Conference Series*, Vol. 2062. IOP Publishing, 012018.

[49] Ziran Shan, Zhipeng Li, and Wenli Song. 2024. Research on Human Posture Recognition Method Based on Deep Learning. *Journal of Mechanics in Medicine and Biology* 24, 02 (2024), 2440010.

[50] Vijeta Sharma, Manjari Gupta, Ajai Kumar, and Deepti Mishra. 2021. Video processing using deep learning techniques: A systematic literature review. *IEEE access* 9 (2021), 139489–139507.

[51] Shubham Shetty, Ebrahim Hirani, Abhir Singh, and Reeta Koshy. 2024. Gesture-to-text: A real-time indian sign language translator with pose estimation and lstms. *Procedia Computer Science* 235 (2024), 2684–2692.

[52] Karen Simonyan and Andrew Zisserman. 2014. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556* (2014).

[53] Hyungwoo Song, Minjeong Shin, Yeonjoon Kim, Kyochul Jang, Jaehoon Choi, Hyunggu Jung, and Bongwon Suh. 2025. AltAuthor: A Context-Aware Alt Text Authoring Tool with Image Classification and LMM-Powered Accessibility Compliance. Association for Computing Machinery, New York, NY, USA, 124––128. doi:10.1145/3708557.3716366

[54] Mingxing Tan and Quoc Le. 2019. Efficientnet: Rethinking model scaling for convolutional neural networks. In *International conference on machine learning*. PMLR, 6105–6114.

[55] Joel Thaduri, S Chakradhar Amingad, Ganesh B Regulwar, Vedha Thirmanpally, and J Goutham Kumar Reddy. 2024. Deep Learning for Folk Dance Classification: An Analysis of ConvLSTM and LRCN Architectures. In *2024 Global Conference on Communications and Information Technologies (GCCIT)*. IEEE, 1–6.

[56] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Advances in neural information processing systems* 30 (2017).

[57] Xuechen Yan, Jing Yang, and Togrol Salami. 2025. Classification of Indian classical dances using MnasNet architecture with advanced polar fox optimization for hyperparameter optimization. *Scientific Reports* 15, 1 (2025), 18624.

[58] Xiaofei Yang, Weijia Cao, Dong Tang, Yicong Zhou, and Yao Lu. 2025. ACTN: Adaptive coupling transformer network for hyperspectral image classification. *IEEE Transactions on Geoscience and Remote Sensing* 63 (2025), 1–15.

[59] Zhendong Yang, Ailing Zeng, Chun Yuan, and Yu Li. 2023. Effective whole-body pose estimation with two-stages distillation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*. 4210–4220.

[60] Rujing Yue, Zhiqiang Tian, and Shaoyi Du. 2022. Action recognition based on RGB and skeleton data sets: A survey. *Neurocomputing* 512 (2022), 287–306.

[61] Joe Yue-Hei Ng, Matthew Hausknecht, Sudheendra Vijayanarasimhan, Oriol Vinyals, Rajat Monga, and George Toderici. 2015. Beyond short snippets: Deep networks for video classification. In *Proceedings of the IEEE conference on computer vision and pattern recognition*. IEEE, 4694–4702.

Fig. 9. *Mudras* used in Kathakali. Pictures of each of the twenty-four *Mudras* are taken separately and joined together to create the figure.

Table 5. List of Mudras commonly used in Kathakali

Mudras used in Kathakali			
Pataaka	*Kapitthakam*	*Mukuram*	*Sarpasirassu*
Mudraakhyam	*Hamsapaksham*	*Bhramaram*	*Vardhamaanakam*
Katakam	*Shikaram*	*Soocheemukham*	*Araalam*
Mushti	*Hamsaasyam*	*Pallavam*	*Oornanabham*
Kartareemukham	*Anjali*	*Thripataaka*	*Mukulam*
Shukatundam	*Ardhachandram*	*Mrigasheersham*	*Katakaamukham*

[62] Ying Zhang, Chenqiong Zhao, Yuan Yao, Chunxiao Wang, Guoliang Cai, and Gang Wang. 2024. Human posture estimation and action recognition on fitness behavior and fitness. *Alexandria Engineering Journal* 107 (2024), 434–442.

[63] Yiwen Zhang, Tao Zhu, Huansheng Ning, and Zhenyu Liu. 2021. Classroom student posture recognition based on an improved high-resolution network. *EURASIP Journal on Wireless Communications and Networking* 2021, 1 (2021), 140.

A List of Mudras commonly used in Kathakali

The *Mudras* commonly used in Kathakali are listed in Table 5 and are also shown in Figure 9.

Table 6. Keyframe distribution across words

Word	Images per word	Images in class A	Images in class B	Images in class C
Aakasham	120	49	35	36
Aana	99	62	24	13
Aayudham	184	45	75	52
Acchan	120	66	28	26
Albutham	175	34	92	33
Amma	183	45	77	46
Anugraham	178	52	59	51
Ariyikkuka	179	67	54	38
Baalan	175	38	53	66
Bhoomi	128	88	27	13
Bhrahmaavu	116	70	25	21
Chandran	107	84	9	14
Chundu	214	52	95	53
Gunam	135	75	14	29
Irikkuka	118	58	31	29
Kaadu	179	64	96	19
Kallan	183	43	92	35
Kannu	243	150	31	37
Kashtam	210	69	91	32
Kuyil	213	84	90	14
Manassu	189	40	79	55
Nadakkuka	105	60	35	10
Nee_Roudram	136	42	80	14
Nee_Shringaaram	108	90	6	12
Paadam	141	73	43	25
Pazham	215	65	46	85
Pushpam	287	85	88	86
Raajyam	244	125	62	35
Samudram	106	69	18	19
Satyam	134	46	73	15
Shivan	98	42	32	24
Simham	68	29	21	18
Sooryan	98	59	27	12
Svargam	142	56	38	48
Thaamara	213	76	98	18
Thalamudi	181	40	85	37
Thaliru	315	59	65	159
Urakkam	162	52	35	57
Veedu	139	42	33	64
Vishnu	124	48	33	43
Vriksham	170	83	20	67

B Keyframe distribution across words

The keyframe distribution across words for each class is given in Table 6

Table 7. Comparison of experimental results with our image classification model results

Model	Accuracy	Precision	Recall	F1-Score
Without correlation	0.8084	0.76	0.76	0.73
Correlation Coefficient 0.95	0.8067	0.75	0.77	0.74
Correlation Coefficient 0.99	0.8134	0.76	0.78	0.75
Pose Multiplication with size 128 (blocks 8 and 9 in Pose Module)	0.8076	0.77	0.78	0.74
CNN Multiplication with size 256 (blocks 10 and 11 in CNN Module)	0.8092	0.76	0.77	0.74
First block in concatenation module (block 1) with size 512	0.8134	0.75	0.78	0.74
First FC layer in Pose module with size 512 (block 5 in pose module)	0.8059	0.76	0.76	0.73
Greyscale images as input	0.8050	0.76	0.77	0.74
Model with filter size 5x5	0.8118	0.74	0.76	0.73
Model with Attention dimension 128 and 16 heads	0.8109	0.76	0.77	0.74
Model with Attention dimension 256 and 16 heads	0.8134	0.75	0.77	0.74
Model with Attention dimension 64 and 4 heads	0.8210	0.76	0.77	0.74
Proposed image classification model	**0.8277**	**0.78**	**0.80**	**0.77**

Table 8. Results of classification with combinations of features

Model	Accuracy	Precision	Recall	F1-Score
Only coordinates	0.7966	0.75	0.76	0.72
Only angles	0.8025	0.76	0.75	0.72
Only finger-to-wrist distances	0.7899	0.73	0.75	0.71
Coordinates and Angles	0.8109	0.77	0.77	0.74
Angles and Finger-to-wrist distances	0.8042	0.75	0.75	0.72
Coordinates and Finger-to-wrist distances	0.8210	0.77	0.79	0.75
Proposed image classification model	**0.8277**	**0.78**	**0.80**	**0.77**

C Results of Experimentations

C.1 Results of experimentations with architecture and pipeline

The results of these experiments are given in Table 7

C.2 Results of experiments with feature combinations

The results of experiments with different feature combinations are given in Table 8